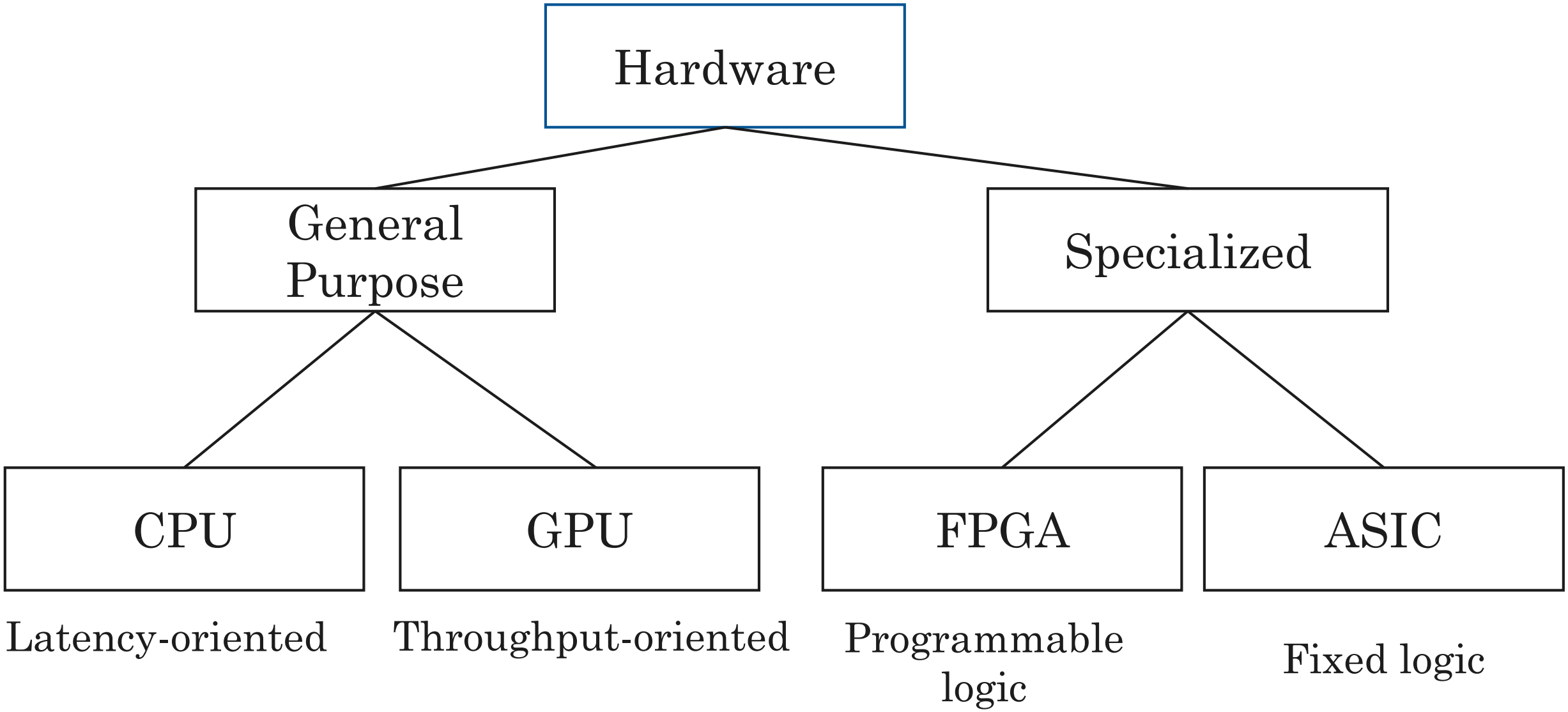
Deep Learning on FPGA

Motivation and Background

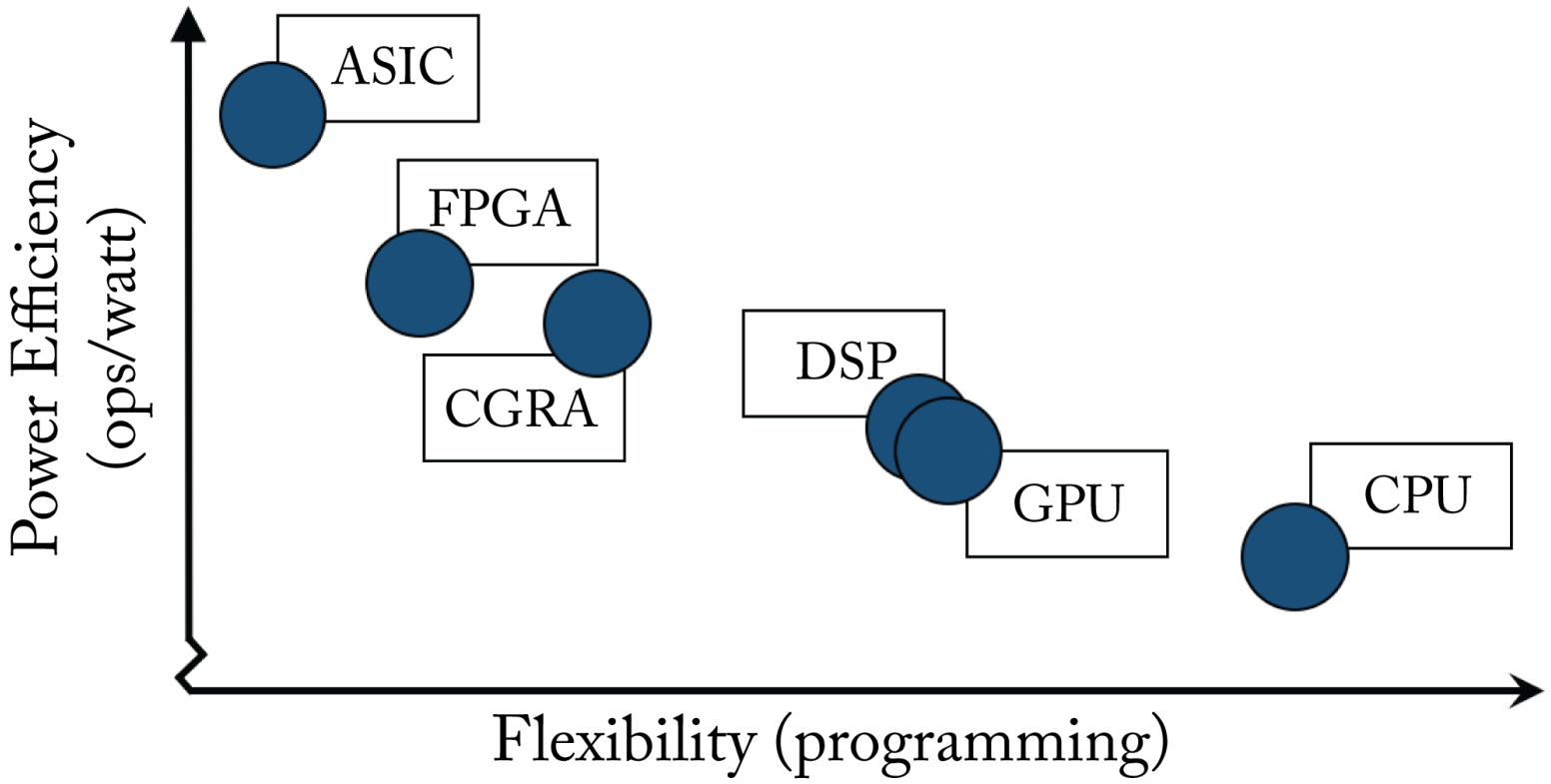
Sparsh Mittal

Acronyms

- NN – Neural network
- DL – Deep learning
- HLS – High-level synthesis
- RTL – register transfer language
- DNN/CNN/BNN – Deep/convolutional/binarized NN
- ASIC – Application-specific IC
- DSP – Digital signal processor
- fmap – Feature map
- BW – Bandwidth
- HW/SW – Hardware/Software
- PE – Processing element
- CONV – Convolution
- FC – Fully connected
- FPGA – Field-programmable gate array
- LUT – Lookup table
- BRAM – Block random-access memory
- MAC – Multiplier accumulator
- GPU – Graphic Processing Unit


Hardware
General Purpose
Specialized
CPU
GPU
FPGA
ASIC
Latency-oriented
Throughput-oriented
Programmable logic
Fixed logic

Tradeoff between processors

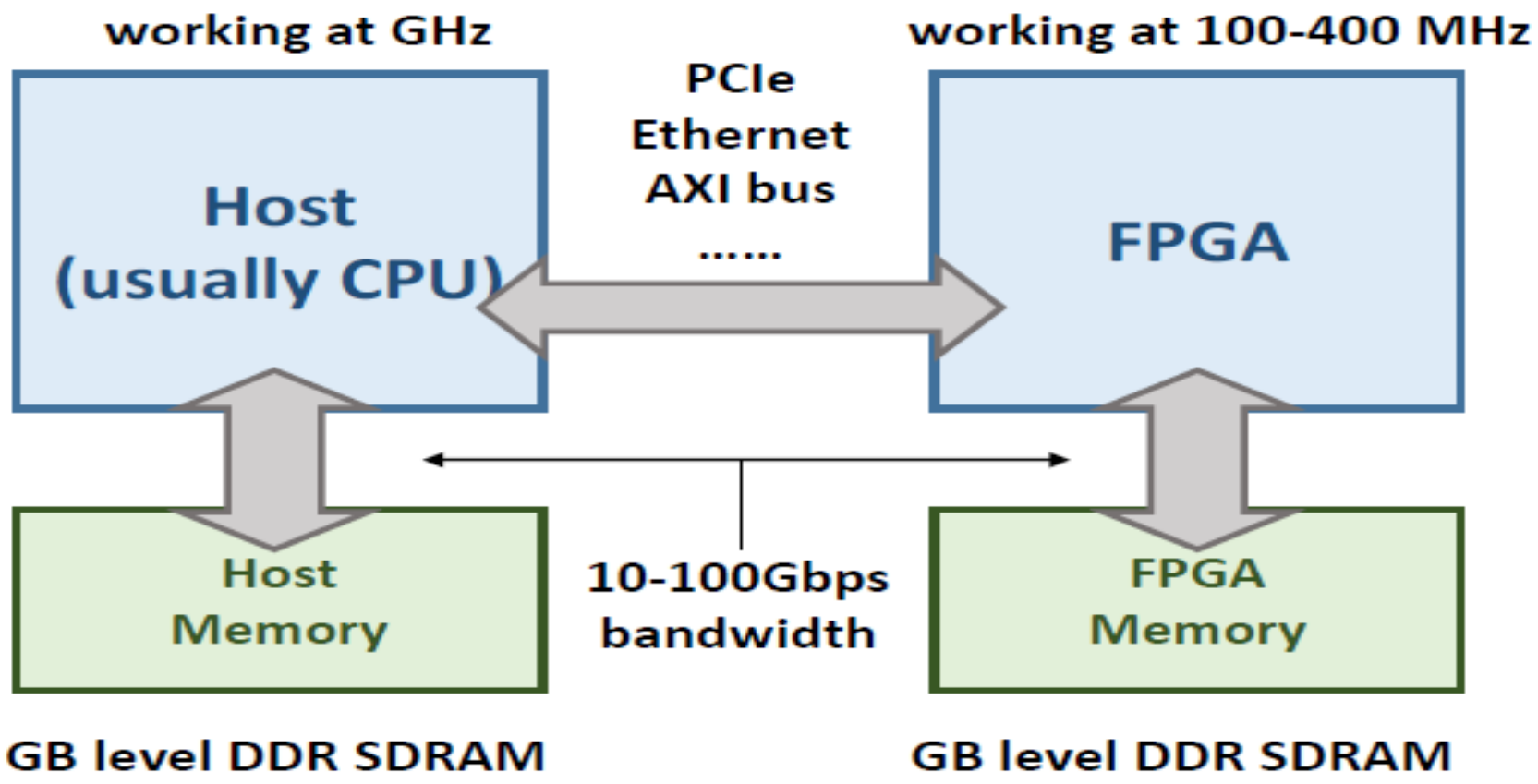


https://deeplearningsystems.ai/#ch07/

A Quick Summary of FPGA Resources

- Four types of resources: DSPs, LUTs, flip-flops and BRAMs.
- DSP: an ALU that can perform logic, add, subtract, and multiply operations. A DSP slice can implement a MAC operation. In Xilinx FPGAs, a DSP slice is termed DSP48 since it can perform up to 48-bit addition.
- LUT: can implement a combinatorial logic operation.
- BRAMs: used for on-chip storage of data.
- FPGAs work at low frequencies (e.g., few 100 MHz) and depend on massive parallelism for achieving high throughput.

FPGA Used as an Accelerator



SDRAM—synchoronous DRAM, DDR—double data rate

Guo, et al. "A survey of FPGA-based neural network accelerator." arXiv preprint arXiv:1712.08934 (2017)

Motivation for Using FPGAs for Deep Learning

- DNN presents high computational demands
- Efficient hardware accelerators are vital for boosting their performance
- CPUs: low throughput and low energy efficiency
- GPUs: high throughput, but lower energy efficiency than FPGA/ASIC
- ASIC
  - allows full customization and small form factor
  - but leads to complex design cycle and high initial investments
  - lack of reconfigurability, which is crucial given rapid improvements in DNN architectures

Challenges in Using FPGAs for CNNs

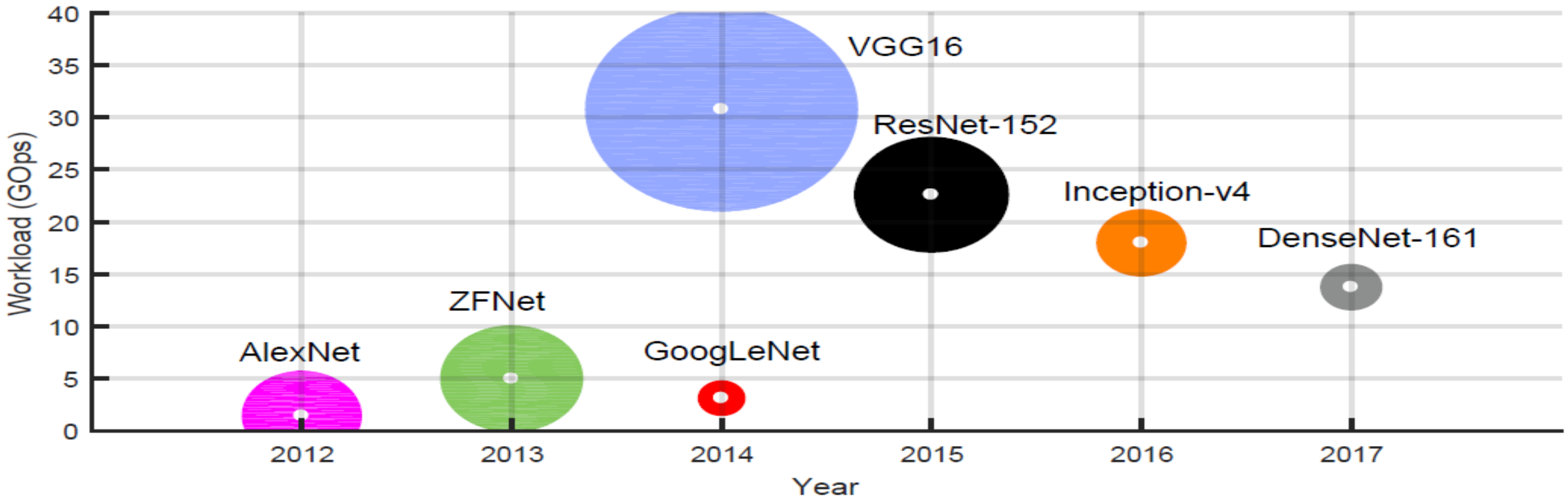
Limited Compute/Memory Resources of FPGA

- For example, second layer of LeNet5 requires 2,400 multipliers, which cannot be provided by most FPGAs.
- FPGA BW << GPU BW (20 vs. 700 GB/s).

- AlexNet has >60 million model parameters. Storing them in 32-b floating point format requires ~250 MB storage space.
- This far exceeds on-chip memory capacity of FPGAs.

- Other networks, such as GoogLeNet and VGGNet, have even higher memory requirement.

Evolution of CNNs over Time

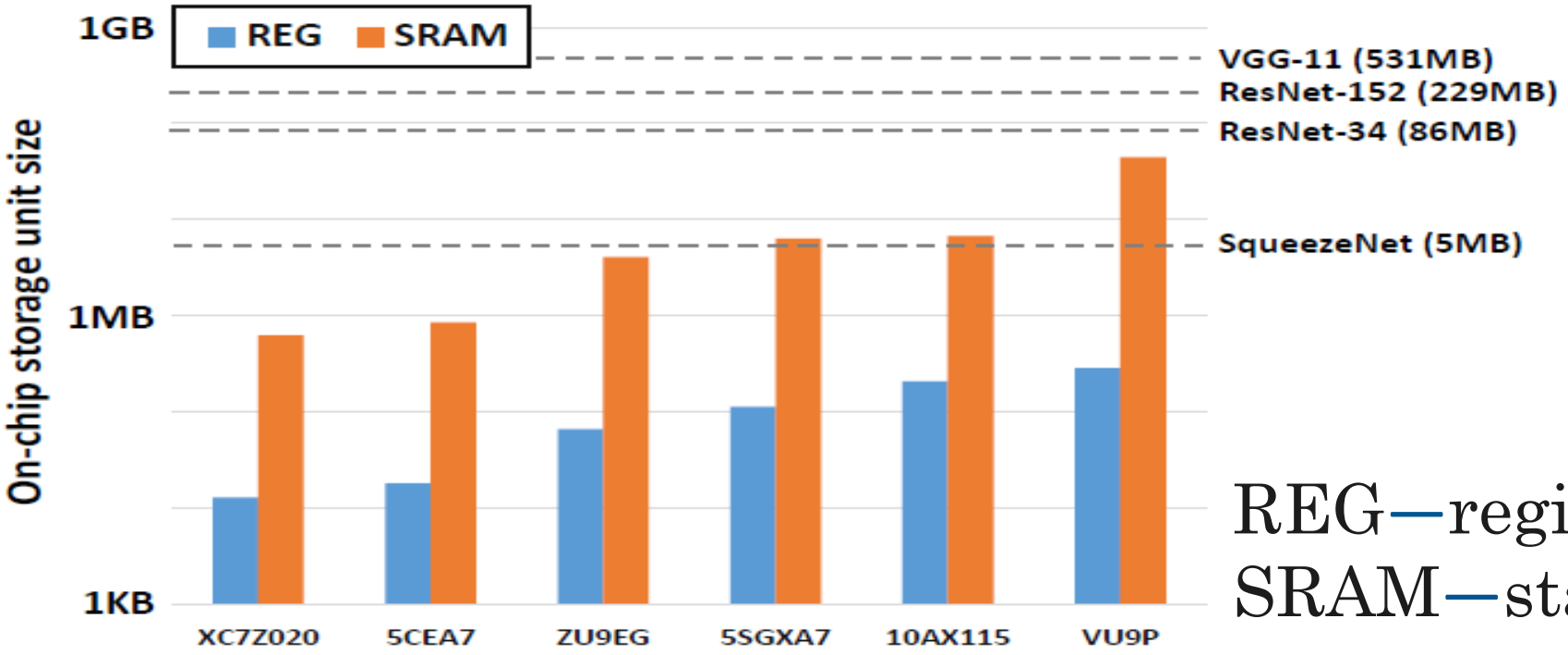


Circle sizes are proportional to the amount of weights

S. I. Venieris *et al.*, "Toolflows for mapping convolutional neural networks on FPGAs: A survey and future directions," *ACM Computing Surveys*, vol. 51, no, 3, May 2019, Art. no. 56.

Limited On-Chip Memory

- Transferring these values to/from off-chip memory leads to performance/energy overheads.
- Off-chip accesses can be reduced by using on-chip memory, but they consume huge area.



REG—register
SRAM—static RAM

Guo, et al. "A survey of FPGA-based neural network accelerator." arXiv preprint arXiv:1712.08934 (2017).

Need of Careful Design Optimization

- Designs with same resource utilization may have vastly different performance ⇒ resource utilization not a good indicator of performance.
- Parameters of different CNN layers (e.g., kernel size, image size, and number of feature maps) are vastly different ⇒ reusing the hardware module for different layers may lead to poor throughput
  - due to a mismatch with computation patterns of different layers and reconfiguration overhead.
- Compute throughput should be matched with memory bandwidth, otherwise one of them will be a bottleneck.

FPGA vis-à-vis Other Computing Platforms

- Tools for developing NN models on CPUs/GPUs are already mature, whereas those for FPGAs are still in early stages.
- Even with HLS tools, implementing CNN model on FPGAs may require multiple months.
- Hence, an FPGA-based design may not efficiently model the most recent NN algorithm.
- In general, compared to FPGAs, GPUs provide higher performance with much lower design effort.

CNN Models and Their Computational Challenges

Model Name		Year	Depth	Design Principles	Challenges
AlexNet	[46]	2012	8	1) Increased depth 2) Increased layer width	1) Non-uniform filter sizes 2) Grouped convolutions
ZFNet	[94]	2013	8	1) Wider layers	1) Computational load 2) Memory footprint
VGG16	[78]	2014	16	1) Increased layer depth 2) Uniform filter size	1) Computational load 2) Memory footprint
GoogLeNet	[83]	2014	22	1) Inception module	1) Irregular computations 2) Irregular layer connectivity
ResNet-152	[33]	2015	152	1) Residual block	1) Irregular computations 2) Irregular layer connectivity
Inception-v4	[82]	2016	72	1) Residual Inception block	1) Irregular computations 2) Irregular layer connectivity
DenseNet-161	[36]	2017	161	1) Dense block	1) Irregular computations 2) Irregular layer connectivity

S. I. Venieris *et al.*, "Toolflows for mapping convolutional neural networks on FPGAs: A survey and future directions," *ACM Computing Surveys*, vol. 51, no, 3, May 2019, Art. no. 56.

CNN-to-FPGA Toolflows

Toolflow Name	Interface	Year
fpgaConvNet [86][87][88][85]	Caffe & Torch	May 2016
DeepBurning [90]	Caffe	June 2016
Angel-Eye [68][23][24]	Caffe	July 2016
ALAMO [58][56][57][55][59]	Caffe	August 2016
HADDOC2 [1][2]	Caffe	September 2016
DNNWEAVER [75][76]	Caffe	October 2016
Caffeine [98]	Caffe	November 2016
AutoCodeGen [54]	Proprietary Input Format	December 2016
FINN [84][19]	Theano	February 2017
FP-DNN [22]	TensorFlow	May 2017
Snowflake [21][10]	Torch	May 2017
SysArrayAccel [91]	C Program	June 2017
FFTCodeGen [100][97][96][95]	Proprietary Input Format	December 2017

S. I. Venieris *et al.*, "Toolflows for mapping convolutional neural networks on FPGAs: A survey and future directions," *ACM Computing Surveys*, vol. 51, no, 3, May 2019, Art. no. 56.

Future Challenges

- Complex blocks break uniform connectivity and computation pattern of traditional CNNs with irregular layer connections.
- Mapping compressed and sparse CNNs to hardware.
- Existing works support Caffe or TensorFlow only. Supporting newer frameworks, for example, MXNet, PyTorch, Caffe2, CoreML, and CNTK.
- Using FPGA for training (not just inference), especially with low-precision training.